

Motion Planning for Multiple Cooperating Manipulators

Kalin Kochnev

The Collaborative Robot Task Sequencing Problem (CRTSP) is a critical challenge in the Fourth Industrial Revolution's push toward mass-customization and autonomous manufacturing. This report aims to give the reader an introduction to methods for combined motion planning and task ordering while optimizing for multiple objectives like collision avoidance and cycle time. The reader is introduced to genetic algorithms and heuristic approaches for tackling these issues.

December 04, 2024

Contents

1. Introduction	3
1.1. Report format	3
1.2. Background	3
1.3. Novel Concepts	5
2. Methods and Technology	6
2.1. The Robot Task Sequencing Problem (RTSP)	6
2.1.1. Overview	6
2.1.2. Classifications of Traveling Salesman Problems (TSP)	6
2.1.3. TSP Solvers	7
2.1.4. Motion Planning Methods	8
2.2. Genetic Algorithms (GA)	8
2.2.1. Introduction to GAs	8
2.2.2. Gene Groups	9
2.3. Application of a Genetic Algorithm for the RTSP	10
2.3.1. Fitness Functions	10
2.3.1.1. Minimum Cycle Time Fitness	10
2.3.1.2. Collision Avoidance Fitness Function	10
2.3.1.3. Combined Fitness Function	11
2.3.2. Results	11
2.4. The Collaborative Robot Task Sequencing Problem (CRTSP)	13
2.4.1. Overview	13
2.5. Application of a Heuristic Method for the CRTSP	13
2.5.1. Novel Concepts	13
2.5.2. Working Points Assignment	14
2.5.3. Resource Allocation	15
2.5.4. Configuration Optimization and Task Ordering	16
2.5.5. Motion Planning	17
2.5.6. Results	17
2.6. Future Work	18
3. Conclusion	19
Bibliography	20
Index of Figures	21

1. Introduction

1.1. Report format

The Robot Task Sequencing Problem (RTSP) is still an active area of research. Only in the past few years has the research community made strides in developing practical solutions to the most general versions of the problem. With the fast-paced nature of this field in mind, the goal of this report isn't to provide a comprehensive review of all the techniques applied to solve the problem. This is because up to this point, many algorithms have been repurposed from other branches of computer science to see what sticks. Some solutions have been more effective than others, and have often been combined to solve different sub-problems.

There has been everything from heuristics, neural networks, constraint solvers, genetic algorithms, simulated annealing, particle swarms and ant colonies, optimization algorithms, a plethora of traveling salesman solvers, optimization methods, and elastic nets, to name a few. Due to the interdisciplinary nature of the CRTSP, it would be impossible to cover all the prerequisites necessary to do justice to this fascinating topic. Therefore I have chosen to focus on genetic algorithms and heuristic approaches because of their applicability to single and multi-robot systems. The reader is assumed to understand the main concepts used in robotics like the kinematics of manipulators and sampling-based motion planning. Everything else will be introduced here.

1.2. Background

Like the introduction of machines for manufacturing during the first industrial revolution in the 1920s, we are also in the middle of a transformational period in automation for the 21st century. We are at the convergence of impressive developments in robotic hardware, perception, big data, cloud computing, and artificial intelligence. All of this will enable a never-before-seen amount of productivity and customizability. During this Fourth Industrial Revolution, we will likely see increased human-robot collaboration and optimization of factory processes and supply chains. There is likely to be a paradigm shift in how goods are produced. We may be able to finally realize a purely on-demand manufacturing service that entirely removes the possibility of miscalculating demand and over-producing.

This paper investigates promising techniques in the field of robotic automation and manufacturing that will accelerate the realization of the Fourth Industrial Revolution. Manipulators are of interest because of their ubiquity in factories and their flexibility in the kinds of tasks they can undertake. Manipulators are frequently programmed to precisely repeat tasks over and over again for rapid duplication. They are exceptional at doing repetitive tasks quickly such as pick and place, welding, assembling, material transport, and handling in factories. However, this loses out on the opportunity to have highly customized workloads. From the robot's perspective, it makes no difference whether it is repeating a motion a million times versus doing a million unique motions.

Additionally, manipulators are often kept in isolated environments away from people, and other robots, because of safety concerns [1]. This decoupling reduces the overall efficiency of the factory. In the future, robots will become more human-centric and closely integrated.

While there is significant interest in robots with collaborative capabilities with humans [1], known as “cobots”, the issue of robot-on-robot collaboration is still an open question. How can robots work together to achieve an end goal? How do the robots decide to distribute the work and how to coordinate with one another?

These questions will likely remain for many years since they are at the root of robotics and artificial intelligence. However, in some specific but highly relevant situations, these questions may be answered sooner rather than later, without answering the question of intelligence.

An impending paradigm shift with the Fourth Industrial Revolution is the transition from mass production to mass customizability [1]. As manufacturing becomes more bespoke, it is imperative to build robots that are capable of making decisions without supervision. And while some of these autonomy capabilities exist today, they mainly exist in robots in isolation. The separation of robots from one another has been a simplifying assumption that is now beginning to be challenged. In the same way that factories parallelized workloads to employees, parallelization of workloads to groups of robots is required to realize the full potential of robotic automation.

Consider two scenarios where collaborative robotics can lead to direct improvements. A printed circuit board (PCB) factory wants to service as many orders as possible, but the customer must meet a minimum batch size requirement. Imagine a future where fully integrated “micro-factories” are capable of working together to service the entire process of PCB manufacturing.

Another possible situation may be a furniture brand selling unique furniture down to the level of the individual order. Existing methods could work. Tools such as multi-axis milling machines are used to manufacture parts in a subtractive manner. A high-speed tool head whittles away material to leave a finished part. These tools can allow for complex geometries and meet the customer’s needs. However, they are typically cost-prohibitive, wasteful, have lower strength-to-weight ratios, and more.

Additive manufacturing has been a rapidly growing technology primarily used in the prototyping of parts during research and development. 3D printing is cost-effective, performant, and highly reconfigurable. The main hurdle to replacing traditional manufacturing methods is scale. 3D printers cannot be scaled as easily as other forms of manufacturing like injection molding. Typically an additive manufacturing system has only a single toolhead, thereby limiting the maximum output. Collaborative robotics is a promising solution to addressing the issue of scaling additive manufacturing.

The single robot task sequencing problem (RTSP) can be boiled down to a robot having a set of known tasks but must figure out the best order in which to execute those tasks. Much of the difficulty in addressing questions related to this problem is related to how broad these tasks are, bordering on Task and Motion Planning (TAMP). Asking a robot to decide how to brew coffee is very different from asking the robot to decide what order to execute a set of trajectories. In a more concrete example, if you have a set of computer chips that need to be placed on the board, what is the best way and order to place those chips? We know where the robot needs to be but not necessarily how or when it will get there.

While a large body of works exists for the Single Robot Task Sequencing Problem, the collaborative extension of this problem, the Collaborative Robot Task Sequencing Problem (CRTSP) has only recently been considered. As mentioned previously, the primary advantage of the CRTSP is the parallelization of workloads. While CRTSP algorithms could be applied at the level of a few manipulators, it can be foreseen that this type of algorithm could be used for higher-level coordination of robots within the factory.

Co-robots must operate at a slower pace because of human interaction [1]. However, a system with simultaneous collaboration between robots can operate faster and reduce cycle time. **Cycle time** is the total amount of time to finish a process from start to finish. However, with collaborating agents, the risk of something going wrong, like a collision, goes up significantly. Not only does an obstacle-free motion plan need to be developed, but it must be developed in a dynamic environment while also optimizing for task ordering.

Because of the combinatorial nature of the problem, standard motion planning search techniques cannot be applied alone. Motion planning algorithms look for a feasible path but do not consider task sequencing or multiple goals. Additionally, the inverse kinematics of the robot can affect the feasibility of a path. So all in all, almost every aspect of robotics must be considered simultaneously in a non-trivial manner: motion planning, inverse kinematics, collision detection, and task ordering. A general framework with minimal approximations or assumptions is desired for this class of problems.

1.3. Novel Concepts

Recent advancements in addressing the Collaborative Robot Task Sequencing Problem (CRTSP) have introduced several novel concepts aimed at improving solution quality. One innovation is the integration of a combined state space for all robots, enabling synchronized motion planning across multiple manipulators. This approach facilitates the consideration of inter-robot dependencies and collision avoidance within a unified framework.

Another development involves the use of gene groups in genetic algorithms, which allow for the encoding of task sequences, inverse kinematic solutions, and intermediate configurations, thereby optimizing multiple objectives simultaneously.

Additionally, heuristic methods have leveraged layered graph representations to model task allocation, resource assignment, and motion planning as interconnected sub-problems in an efficient manner.

These methods, combined with collision-aware fitness functions and resource constraints will be the foundation for future work in this field.

2. Methods and Technology

2.1. The Robot Task Sequencing Problem (RTSP)

2.1.1. Overview

In industries that make heavy use of robot arms, such as automotive manufacturing, operators are tasked with hard coding solutions to these problems, often with suboptimal solutions [2]. While the time investment in hand-tuned solutions may make sense if the robot must repeat the tasks thousands or millions of times, this is untenable for highly customized processes like 3D printing, further highlighting the importance of this issue.

The robotic task sequencing problem is analogous to the Traveling Salesman Problem (TSP), which is an NP-Hard [3]. A solution to the Traveling Salesman Problem is a minimum-cost tour through a set of nodes on a graph so that every point is visited once and returns to the start. In literature, the Shortest Sequence Problem (SSP) and the TSP are often mentioned together. The primary difference between the two problems is that the TSP considers a complete tour while the SSP does not require the salesman to return to their starting point [4].

In comparison to the TSP, the RTSP is non-trivial. It has additional complicating factors such as obstacle avoidance, multiple solutions to the inverse kinematics problem if a manipulator is redundant, task dependence/partial ordering of tasks, and finally, other objectives that may need to be optimized like time or energy. In a literature review of the field by Alatarsev et al., many papers made simplifying assumptions that boiled down the RTSP problem to a TSP problem by ignoring redundant kinematics or collisions [3]. While this may work with simple, single manipulators, this clearly cannot be extended to the cooperative problem.

2.1.2. Classifications of Traveling Salesman Problems (TSP)

Task ordering underpins the RTSP. There are four different categories of RTSP problems depending on the inputs and outputs of the problem [3]. The input is either what is termed a primitive task or a complex task. Meanwhile, the output could be a sequence of tasks or a trajectory to be followed depending on the type. Each combination of input and output corresponds to a different variation of the TSP.

A **primitive task** is a task whose action can be represented as a single point or a finite set of points. For instance, a single-point task might be drilling a hole at a location or a task with a finite set of points may be a welded joint along the side of a part. A **complex task** is a task described by a general region to visit. For instance, a camera inspection robot may be able to visit close enough to a goal instead of requiring an exact location.

In some cases, knowing the sequence of tasks is enough to infer the trajectory of the solution. In other cases, trajectory information becomes relevant when considering complex tasks that do not have exact points to visit. A trajectory that specifies at what points to enter and exit regions is necessary for complex tasks.

This paper primarily focuses on the primitive task input and the trajectory output since it has the broadest application. Sequence is not enough information when considering multiple collaborating robots.

There are additional TSPs that are relevant to studying task ordering. The Asymmetric TSP has the cost to go from one task to another depending on the direction traveled. This may often arise because of robot dynamic constraints where motions may be faster depending on the configuration of the arm. Another variation is the Sequential Ordering Problem (SOP) where the order in which tasks are executed matters. Think of it as a dependency graph where one operation can't be completed until the blocking operation is cleared.

The variations of the traveling salesman problem are summarized in Figure 1

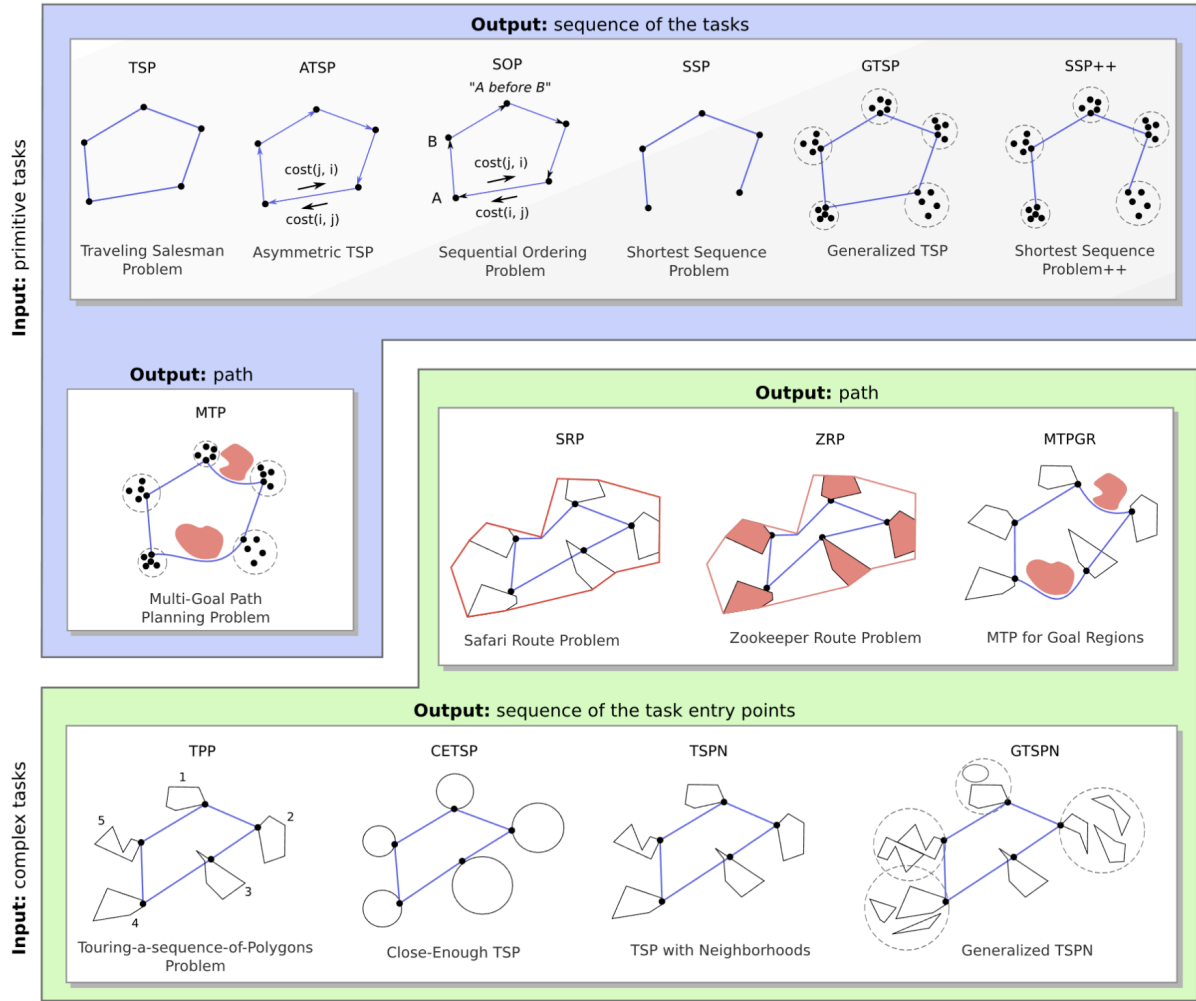


Fig. 3 Sequencing problems are categorized according to their input and output parameters. The tour/path is denoted with blue and obstacles are denoted with red. The dashed circles denotes the clusters

Figure 1: Overview of Traveling Salesman Problem variations from [3]

2.1.3. TSP Solvers

The traveling salesman problem is NP-hard. This means if $P \neq NP$ then the problem cannot be solved in polynomial time. Mixed-Integer Non-linear Programs (MINLP) are specialized solvers to find the optimal solution to a TSP. These solvers are not practical for more than a handful of tasks. In many cases, the most optimal solution is not worth the extra computation, so heuristics are applied to finding a complete tour that is decent. Some popular TSP heuristic algorithms are Insertion Heuristics, 2-Opt, 3-Opt, and Tabu Search [3]. While the heuristic algorithms are effective, their main drawback is that they do not indicate how far from optimality they are.

As mentioned previously, even if the TSP were solved optimally, it is rarely enough to give an adequate solution to the RTSP.

2.1.4. Motion Planning Methods

Sampling-based methods remain relevant for the motion planning component of RTSP. Contrary to other fields of robotics, probabilistic roadmaps are more popular than RRTs due to their multi-query capabilities for offline planning. Artificial potential fields are occasionally used but are avoided due to local minima.

2.2. Genetic Algorithms (GA)

One of the most successful approaches for single manipulators, as well as multiple manipulators, has been using genetic algorithms. These algorithms are advantageous for their ability to encode additional information into the “chromosome” to optimize multiple factors simultaneously, such as solving the traveling salesman problem as well as considering other factors like collisions between robots.

It has been shown that GA algorithms can work well even for NP-Hard problems [2]. The main drawback of genetic algorithmic approaches is computation time. It can be quite time-consuming to run these algorithms compared to some alternative approaches that were developed.

2.2.1. Introduction to GAs

Genetic algorithms are intuitive to understand because of their roots in natural selection in real life. Organisms with the most successful chromosomes are propagated and explored further while unsuccessful organisms are culled.

There are a few key terms necessary to understand how genetic algorithms work. A **gene** is an abstraction that can encode a wide variety of information. A gene usually represents one type of thing, almost like a column in a spreadsheet. It could be a single bit, a character, or a number. Meanwhile, a **chromosome** is a collection of genes that encodes all the information necessary to describe a solution to a problem [4].

For example: Define a gene that consists of 4 character options **A**, **B**, **C**, **D**. A 10-length chromosome has 10 genes whose sequence looks like **BAAAABCD**CD.

The **fitness** of a specific chromosome indicates how good of a solution it is to the problem, as evaluated by a **fitness evaluation function** [4].

New chromosomes are introduced by randomly combining genetic information from multiple parent chromosomes. More than two parents are possible [5]. This is known as **crossing over** [4]. Two common methods for selecting parent chromosomes are random selection or selecting based on fitness (better solutions are selected more). The order of the gene types remains the same but the specific value of a gene may vary.

Mutation is used to inject randomness to increase genetic diversity and solution quality. There are important considerations for how to insert randomness such as decreasing probability of significant changes [5].

These specific processes are used to improve upon solutions without getting trapped in local minima. It has been shown that “seeding” the starting chromosome with a good starting guess can significantly improve the results [2]. It is difficult to tell when to terminate a genetic algorithm, but it usually executes until a certain number of iterations or satisfactory performance is achieved [2]. The ability to explore the search space for multiple objectives makes genetic algorithms a good candidate for RTSP problems.

2.2.2. Gene Groups

In literature, several applications of genetic algorithms for the RTSP problem introduce a new concept known as a gene group [2], [4], [6]. A **gene group** is an additional abstraction on top of a gene. It permits a sequence or list of genes compared to a single gene. This can also be thought of as nesting chromosomes together.

For instance, a single gene with four options A, B, C, D might be combined into a gene group, or chromosome if you prefer, called X . Another gene with four options E, F, G, H might be combined into a gene group/chromosome called Y . The whole chromosome C can be represented as $C = (X, Y)$. But X could be a nested chromosome/sequence of genes $X = (D, D, B, A)$ and $Y = (E, F, F, E, H)$. X and Y have no restriction on the sizes as long as it is consistent between executions.

In the context of RTSP, as an example, let one gene group S represent a sequence of 10 tasks whose order encodes the solution to the TSP.

Another gene group C encodes for each task which of the $[1, n]$ possible inverse kinematic solutions are used. The last one, I is a gene group within a gene group that encodes intermediate configurations of a robot arm to transition from task to task. Let the combined chromosome/gene group be:

$$\text{Chromosome} = (S, C, I) \quad (1)$$

Let $I_{i,j}$ represent the intermediate configurations from task i to task j where $i, j \in (0, 1, 2)$.

$$\text{Chromosome} = ((S_2, S_0, S_1), (C_2, C_0, C_1), (I_{2,0}, I_{0,1})) \quad (2)$$

To give a concrete example from Zacharia et al. [2], the gene group $S = (3, 1, 2, 4)$ encodes the sequence in which to visit tasks $M = 4$ tasks. The next gene group C encodes in binary what configuration to use for the manipulator for each of those M tasks. They restricted the inverse kinematic solution for the 6 DOF manipulator would have only 8 alternative configurations corresponding to each task point to minimize the search space [2]. So for each task, a new gene is encoded to represent which of the 8 inverse kinematic configurations to use for that task in binary $C = (010, 011, \dots, \dots)$

Finally, the last set of genes contains intermediate configurations between tasks to ensure that collisions don't occur during transitions from task to task. In the figure, the intermediate configurations to transition from task 3 to 1 are $I_{3,1} = ((1.23, 0.18, 1.42, 1.11, 2.06, 1.67), (\dots))$. The number of intermediate configurations to use is user-adjustable so that more accuracy can be requested. Additionally, the intermediate configurations are not restricted to 8 inverse kinematic solutions so collision avoidance is more likely.

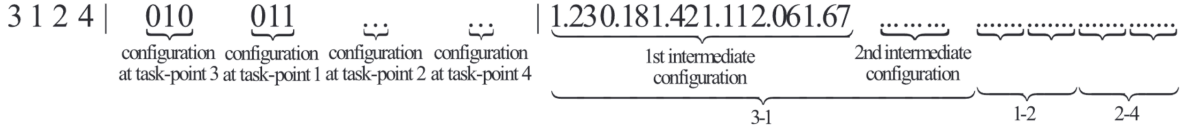


Fig. 3. A possible chromosome.

Figure 2: An example chromosome from [2]

2.3. Application of a Genetic Algorithm for the RTSP

One study [2] makes use of genetic algorithms. Their paper made several contributions to the study of RTSP problems. Their algorithm was capable of determining a feasible motion plan as well as optimizing the task sequencing problem. The combination of motion planning and sequencing enabled them to avoid the use of additional motion planning algorithms in their situation but does not apply in more complicated scenarios. Additionally, they had a novel application of B-spline surfaces to approximate both convex and non-convex shapes efficiently. The B-spline surface was generated based on discretized 3D space where the q entry of four-dimension vector (x, y, z, q) indicates a binary value of whether an obstacle is present in that cell. The B-spline surface representation can be rapidly constructed and queried for fast collision checking.

2.3.1. Fitness Functions

2.3.1.1. Minimum Cycle Time Fitness

The fitness function in Zacharai et al. sought to minimize the path length of the task tour in configuration space (C-space) as a way to minimize cycle time [2]. The time it takes for a robot to transition between two tasks is ultimately dictated by the joint that takes the longest to move [2].

So knowing the change in angle and the average velocity $\dot{\theta}$ for joint n we can find the time cost of a movement in configuration space.

$$t_i = \max_n \left(\frac{|\theta_n^i - \theta_n^{i-1}|}{\dot{\theta}_n} \right) \quad (3)$$

Therefore the total time to complete all tasks R is

$$t_{\text{total}} = \sum_{r=0}^R \max_n \left(\frac{|\theta_n^r - \theta_n^{r-1}|}{\dot{\theta}_n} \right) \quad r = 2, \dots, R \quad (4)$$

Another summation is necessary if you consider intermediate configurations but it is easy to extend the definition. Minimizing this quantity is one of the two fitness functions involved.

2.3.1.2. Collision Avoidance Fitness Function

Another objective function is necessary to prioritize collision avoidance for the end effector.

If a manipulator is tracing a path along the surface of an object, it should be along the B-Spline surface. If the manipulator veers off course perpendicular to the surface, then you know a collision has occurred. Therefore the collision-free trajectory is the one that minimizes the

difference between the B-Spline and the tool-head. Put another way, the objective is to have the most flat trajectory relative to the surface.

In the paper, they formulate the problem slightly differently. Instead of minimizing the difference between the end effector only, they sample points along the manipulator links and minimize the flatness of all sampled points relative to the surface. This prevents collisions with each of the robot links. This fitness function is evaluated for all intermediate configurations while transitioning between tasks to prevent collisions while transitioning tasks.

2.3.1.3. Combined Fitness Function

The overall fitness function is a linear combination of the fitness functions for collision avoidance and cycle time minimization. Each fitness function is weighted so that $w_1 + w_2 = 1$. θ is the set of all configurations from start to finish, including the intermediate configurations to complete all tasks.

$$E(\theta) = w_1 t_{\text{total}}(\theta) + w_2 H(\theta) \quad (5)$$

The multi-objective function is minimized since we want to minimize cycle time and minimize the variation along the B-spline surface.

$$\min_{\theta}(E) = \min_{\theta}(w_1 t_{\text{total}}(\theta) + w_2 H(\theta)) \quad (6)$$

As described in the genetic algorithm overview, the configurations for the inverse kinematics are limited to one of eight different solutions for the task point solutions. This is to decrease the search space wherever possible. However, the intermediate configurations between tasks are permitted to have any number of solutions to avoid collisions.

2.3.2. Results

To quantify the performance of this method the authors ran several scenarios to assess the quality of the solutions. They found that the algorithm was capable of finding near-optimal solutions in terms of cycle time optimization. However, CPU time would grow quickly which implies this method is not suitable for a high volume of tasks.

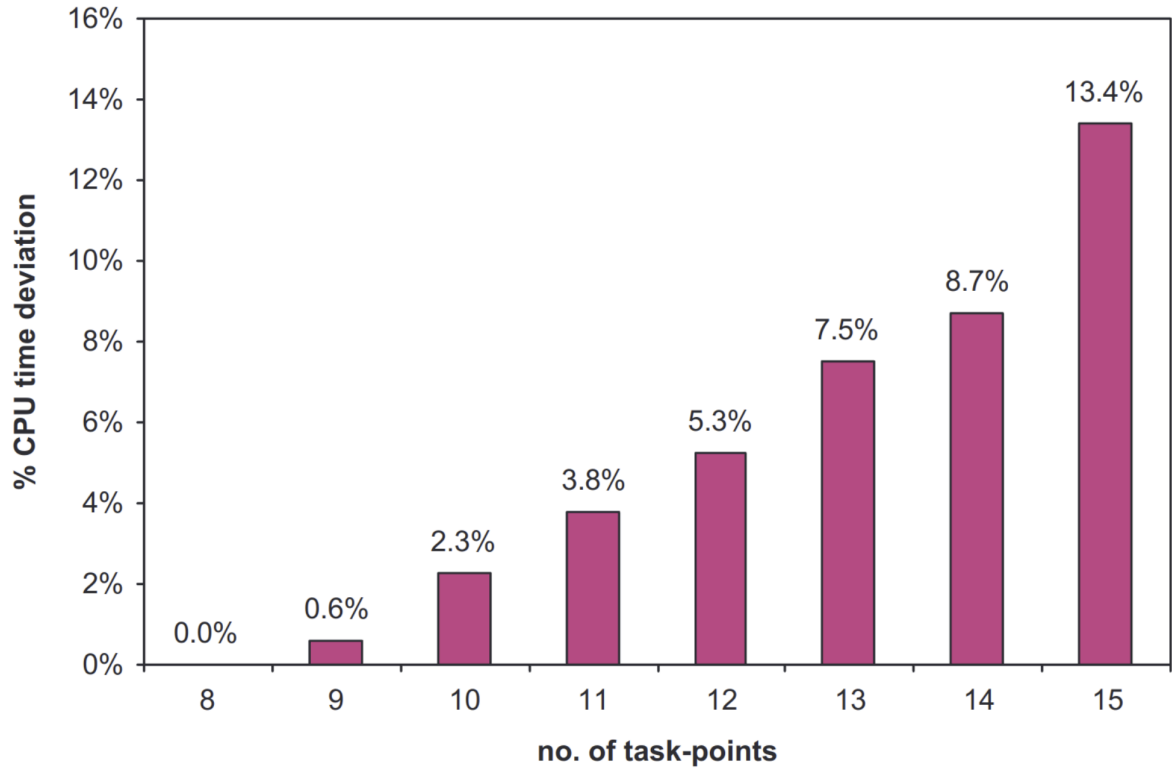


Fig. 13. The effect of task-points in CPU time.

Figure 3: Relative CPU time growth from [2]

They compared their collision detection with B-Splines using a minimum-distance detection by bounding boxes method. They found that their method of optimizing for collision avoidance resulted in fewer generations to reach the locally optimal solution than the alternative method.

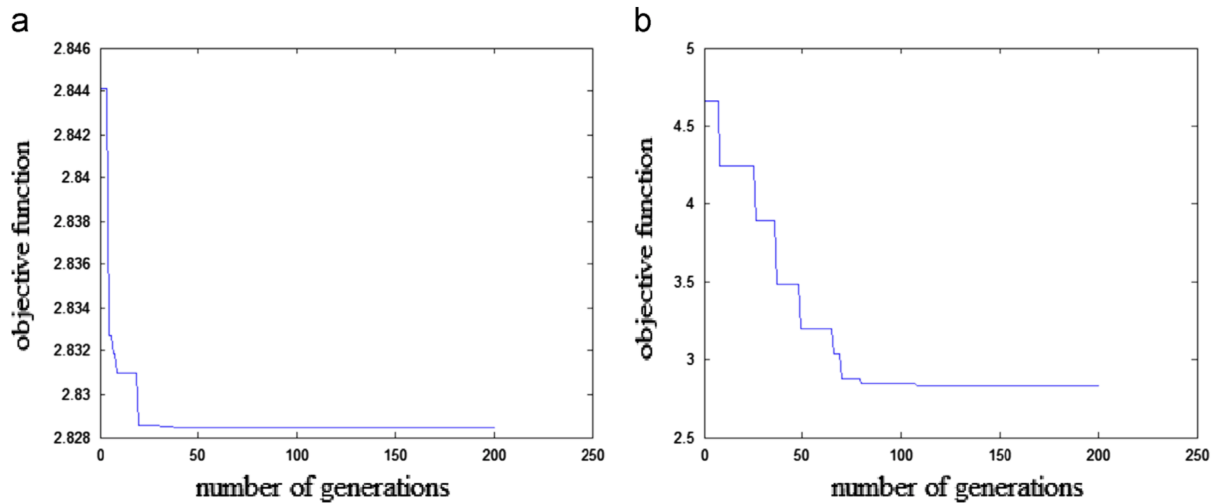


Figure 4: Objective function over a number of generations from [2]. (a) is the B-Spline collision detection while (b) is the bounding box method

2.4. The Collaborative Robot Task Sequencing Problem (CRTSP)

2.4.1. Overview

The Collaborative Robot Task Sequencing Problem has many additional difficulties on top of the RTSP. The main questions are how to prevent collision between multiple robots and how to optimally distribute work across multiple agents.

The CRTSP is related to the multi-robot task allocation (MRTA) problem [7]. The MRTA problem decides how to allocate a set of tasks among multiple robots to meet an overarching goal. The main difference between the MRTA and the RTSP is that the RTSP considers how collisions will be avoided as well as how tasks are allocated.

There have been a few approaches to solving the collaborative RTSP. One approach is to partition the workspace into regions accessible to certain robots [8]. This eliminates the need to provide synchronization signals between robots. However, this potentially reduces the cycle time of the robots because the arms cannot cross another arm's workspace, even if it would be beneficial.

Xin et al. approach uses a Time-Space Network Model (TSN) which considers time and space constraints together and successfully solves the problem [9].

Touzani et al. use a genetic algorithm whose chromosome contains all tours and improves solutions in that manner [6]. Spensieri et al. use a branch and bound (B&B) technique to solve the min-max Multiple Generalized Travelling Salesman Problem (min-max mGTSP) without collisions and then refine the min-max mGTSP with an iterative procedure to avoid collision [10]. Their approach also allows for synchronous operations where each robot completes its tasks simultaneously or in asynchronous modes. Li et al. consider resource constraints and multi-robot tasks by building upon solutions to the sub-problems [7].

2.5. Application of a Heuristic Method for the CRTSP

2.5.1. Novel Concepts

A deeper dive into the methods of Li et al. is considered [7]. They compare their result with a genetic algorithm developed in [6].

It is a common situation for robots to need access to shared resources like a part or a tool. Unfortunately, this problem cannot simply be solved by a TSP since it is not as simple as finding an optimal tour for those tasks. The order of the tour must respect that certain resources must be allocated prior to working on a task. There are additional constraints such as ensuring that all resources g_i used for all robots k are independent of each other ($\cap_{\forall k} g_i^k = \emptyset$), all working points are visited to complete a task. ($\cup_{\forall k} p_i^k = P_i$), and that no collisions are present.

There are a few concepts that are commonly referred to in the paper. There are N_g resources, N_t robots, and N_t tasks.

This paper takes a different approach than the genetic algorithm previously described. Each of the related problems for the RTSP is broken down so that they can be solved separately, and later on, refined and adjusted to yield a feasible solution.

The general procedure of the algorithm is

1. Assignment of working points to robots using Dijkstra variation.
2. For every allocated working point, assign resources to them.
3. Find the best configurations to visit the working points and optimize task sequence based on distance in configuration space.
4. Determine the motion plan from point to point.

2.5.2. Working Points Assignment

The issue of how to distribute working points across robots is modeled as a graph problem where the vertices are working points and the edges are a feasible transition from working point to working point. The weight of a particular edge indicates the cost to move from one working point to another, such as time or distance.

A tour S^k along the graph for robot k is defined as visiting a sequence of working points s_i starting from a home position p_0^k and eventually returning the home position. The total cost to complete this tour $C(S^k)$ can easily be computed by summing the edges along the tour. The goal of the working points assignment is to give each robot its own tour through the working points. The set containing the tour of each robot is $S = (S^1, \dots, S^{N_r})$ where there are N_r robots.

The goal of the algorithm is to minimize the cost of the most expensive tour.

$$\text{Cost}_S(S) = \max_k (C(S^k)) \quad (7)$$

The graph problem is reorganized in order to develop an assignment of working points. Each layer corresponds to a single multi-robot task of which there are N_t multi-robot tasks (there are N_r robots). The multi-robot task layers are ordered according to the order defined by the TSP solver. The individual layers are composed of all the working points that correspond to one multi-robot task.

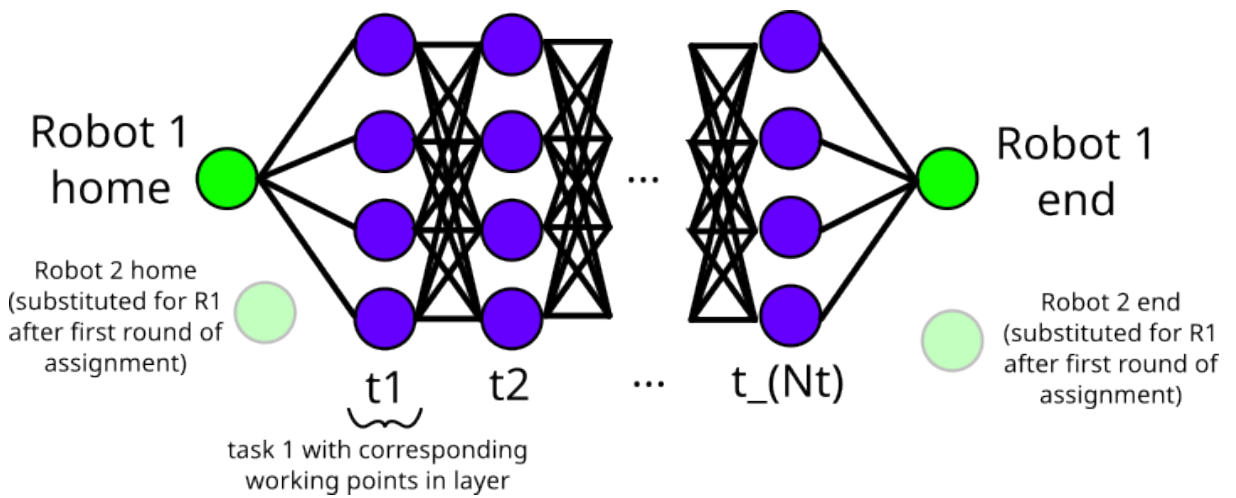


Figure 5: Working point assignment graph structure

Each layer is fully connected with the layer preceding and after itself. However, vertices (working points) in the same layer are unconnected. For each robot, its starting and ending position is fully connected to the starting and ending layers, respectively.

To finalize the assignment, Dijkstra's algorithm is applied for each robot to find the shortest path from the home and end position. Once a working point tour has been assigned for a robot, the edges and vertices corresponding to the tour are removed from the graph before the next tour is computed for another robot. Dijkstra's algorithm is repeated until all robots have been assigned a tour.

The final step in the assignment algorithm is to check for collisions along the tour. If there is a collision, the working points that correspond to the collision must be changed. The paper does not specifically outline a method on how to do this but they specifically mention that the colliding working points must be swapped with a working point within the same task (same layer). So swapping of working points may have to occur until a collision is not caused.

2.5.3. Resource Allocation

At this point, the optimal tours were assigned. However, this does not address the issue of conflicting resources. The paper models this as a constraint assignment problem. A constraint assignment problem is determining how to assign a number of tasks to a number of agents with certain constraints.

Several constraints are imposed on how resources are allocated.

1. Every working point gets only one resource.
2. One resource is employed by one robot at a time
3. Obtaining a resource cannot result in a collision

A flag variable $x_{grt} = 1$ and 0 otherwise if a resource g has been allocated to robot r for task t . c_{grt} indicates the cost to assign resource g for robot r and task t . The cycle time metric is the worst-case cycle time across all robots for all the assignments made

Finding an optimal solution to this problem is difficult, but can be solved in a similar way as the working point assignment problem. A layered graph is constructed where each layer represents a working point s_i . The order of the layers/working points is determined by the working point tour S^k for robot k . The nodes within a layer represent all possible resources g that could be allocated for that working point. Edges in consecutive layers represent the transitions between resource assignments for adjacent working points. The edge weights are the cost c_{grt} of switching from one resource to another.

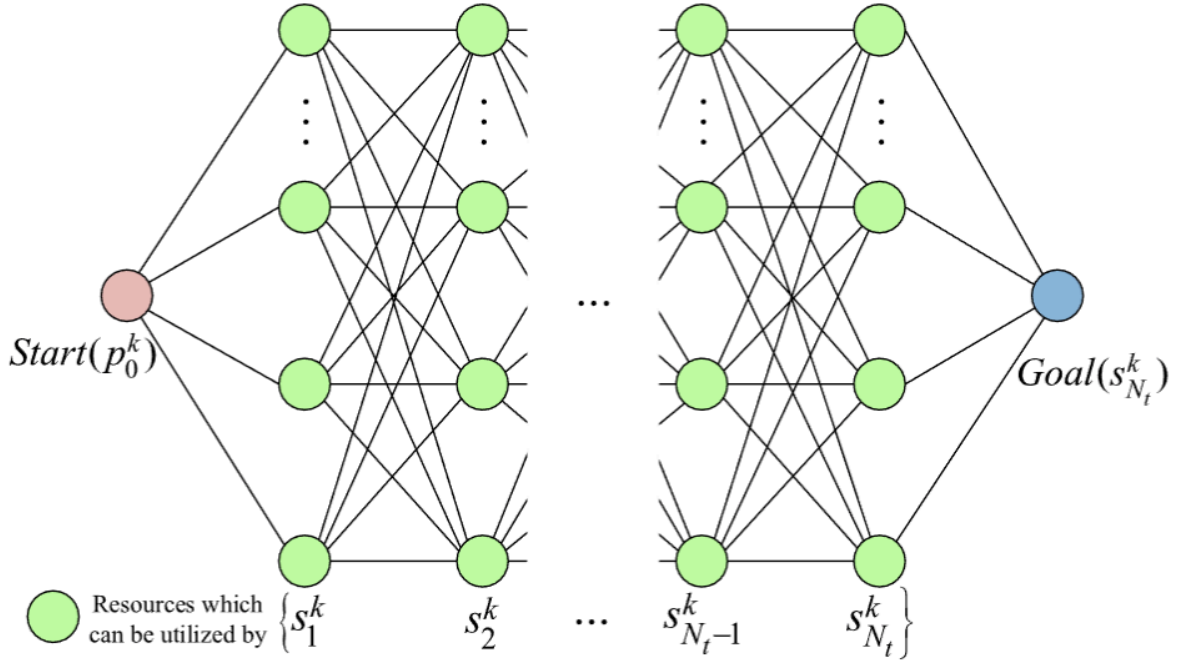


Figure 6: Resource allocation graph from [7]

To ensure optimal allocation, Dijkstra’s algorithm is applied to find the shortest path through this graph. The “Start” node is connected to all resources in the first task layer, representing the robot’s starting point, and the “Goal” node is connected to all resources in the last task layer, representing the end of the sequence. By finding the shortest path from “Start” to “Goal,” the algorithm identifies a sequence of resource assignments that minimizes the cumulative cost while respecting constraints.

It is very important to realize that this graph corresponds to a *single* robot k . Once an assignment of resources has been determined for robot k , the resource/working point pairs allocated for robot k need to be pruned from the graph so that for a different robot it is not possible to allocate the same resource for the same task. After pruning resources, the graph is reconstructed for a different robot and its tour.

2.5.4. Configuration Optimization and Task Ordering

Since a redundant manipulator can have multiple inverse kinematics solutions, a way to choose between solutions is necessary. Li et al. define a metric $\Phi(q)$ that penalizes inverse kinematic solutions that are in the opposite direction of rotation as the home configuration. This is presumably to minimize jerky motion back and forth as well as to incentivize solutions near the home configuration. Each joint is weighted differently with ω_i

$$\Phi(q) = \Phi(q, q_0) = \sum_i^{\text{DoF}} \omega_i (q(i)q_0(i) - |q(i)q_0(i)|) \quad (8)$$

The task ordering of a tour is solved using the 2-Opt algorithm for the TSP problem. They define the cost of going between tasks in terms of configuration space. The worst-case cost across all robots to transition from task t_i to t_{i+1} in configuration space is considered. Q_i^k corresponds to the robot configurations for the working point p_i and resource g_i for robot k .

$$Q_i^k = (q_{g_i}^k, q_{p_i}^k) \quad (9)$$

$$\text{Cost}(t_i, t_{i+1}) = \max_{k=1 \dots N_r} C_Q(Q_i^k, Q_{i+1}^k) \quad (10)$$

Since a resource must be acquired before starting a task, the total task transition cost can be approximated as the cost $c(\cdot)$ to switch between resources g_i, g_{i+1} and the cost to go between the tasks p_i, p_{i+1} , which can each be weighted with ω_g, ω_p . $c(\cdot)$ estimates the travel time in configuration space using Equation 3 from earlier.

$$\begin{aligned} C_Q(Q_i^k, Q_{i+1}^k) &= \omega_g c(q_{g_i}^k, q_{g_{i+1}}^k) + \omega_p c(q_{p_i}^k, q_{p_{i+1}}^k) \\ c(q_i, q_{i+1}) &= \max_n \left(\frac{|q_i - q_{i+1}|}{\dot{q}} \right) \end{aligned} \quad (11)$$

2.5.5. Motion Planning

The configurations of multiple manipulators are combined into a single state space where one state specifies the configurations of all manipulators. A Bi-RRT algorithm is applied for the motion planning in this combined space.

A feasible initial state is determined and built by sampling collision-free configurations in the combined space. Once the initial state and the goal state have a path found, the motion planning is complete between the two states. Edge cases like no solutions are handled by adding delays to a robot or modifying the geometric path from point to point.

2.5.6. Results

Li et al. compared their algorithm performance with [6] as well as their own rule-based approach that provides a feasible solution. The presented algorithm has a lower cycle time than the min(sum-max) mGTSP while also being about two orders of magnitude faster. Additionally, this algorithm has a slightly faster run time than the rule-based approach while showing a massive improvement over the rule-based approach.

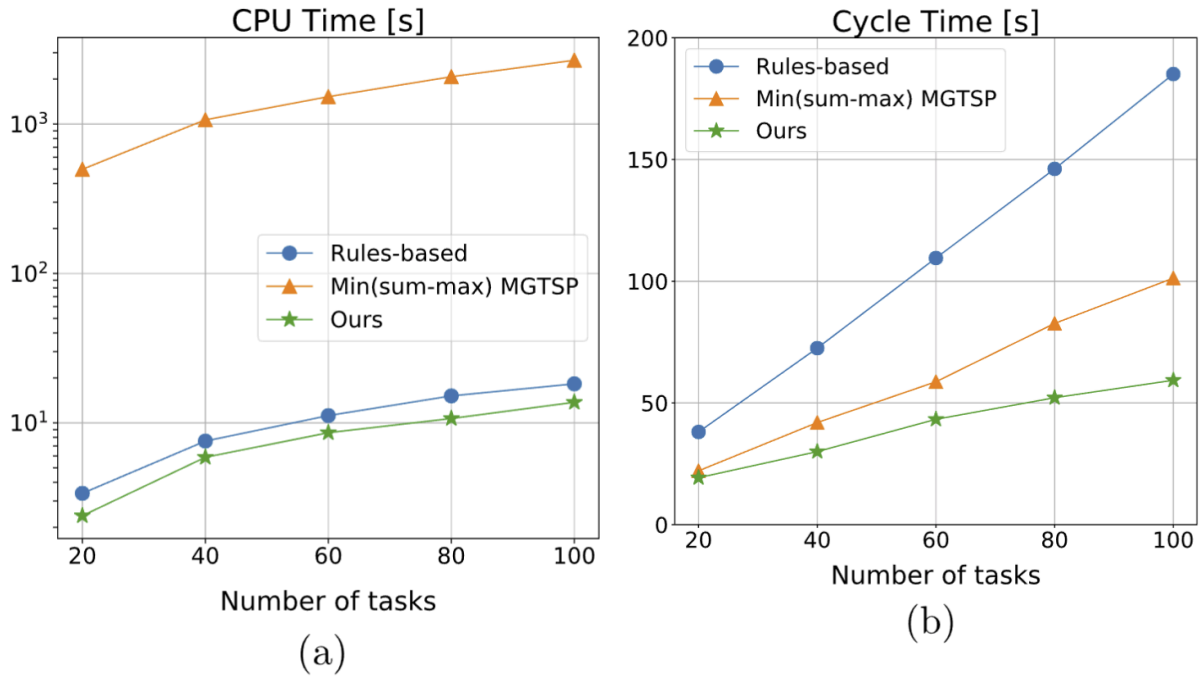


Figure 7: Results of the heuristic method in [7].

(a) CPU runtime versus number of tasks. (b) Cycle time versus number of tasks

2.6. Future Work

This area of research is still in its beginning stages. And there are many potential opportunities for optimization especially when it comes to developing a general framework capable of working for multiple industrial processes.

There has been recent progress in the collaborative robotic task sequencing problem with redundant kinematics. The problem is far from solved. When multiple configurations of robot arms can be evaluated this may allow for solutions to the RTSP that would not have been possible otherwise, especially in scenarios with multiple robots with closely overlapping workspaces. Algorithms that can take advantage of the multiple inverse kinematic solutions can increase the synergy of multiple robot arms with closely overlapping workspaces and increase productivity.

In almost all instances to date, robot dynamics have not been considered in the problem formulation. Without this, the calculated costs used in the traveling salesman problem are rough approximations and don't consider many other factors like energy minimization.

Related to dynamics is trajectory information. It is okay for multiple robots to have the same point as long as it is not causing a collision for the times the robot is at that point. Many of the approaches do not consider how parameterizing a collision-free path with time can be used to avoid a collision. This is a potential area of improvement.

When it comes to solving the traveling salesman problem the dependence of task order has not been considered. And finally, there is not a standard benchmark to be used for comparing different algorithms.

3. Conclusion

The Collaborative Robot Task Sequencing Problem (CRTSP) is at the center of challenges inherent to modern manufacturing and robotics. The ability for robots to work collaboratively in dynamic, multi-tasking environments is critical for the future of customizable and scalable manufacturing. This report explored key methods for addressing the RTSP and CRTSP, emphasizing genetic algorithms and heuristic approaches. These techniques seek to generate feasible and high-quality solutions for task sequencing, collision avoidance, and cycle time within a reasonable amount of time.

Genetic algorithms have proven themselves to be effective for solving the CRTSP. This is because they can encode multiple objectives and avoid local minima but have issues with scaling. Heuristic methods on the other hand provide faster, more computationally efficient solutions, albeit at the cost of having a general framework for objective optimization.

The research area is still in its early stages, with many areas ripe for further study. One open question is how to integrate robot dynamics into cost and motion planning calculations. Dynamics and task cost are closely linked which is at odds with the decoupling of sub-problems applied by many heuristic methods. Without this, the solutions remain rough approximations. Another challenge lies in the lack of benchmarks for CRTSP algorithms which could further accelerate research in this area. There are areas that have potential for optimization. For instance, the ability to exploit multiple inverse kinematic solutions could enhance the coordination of multiple robots sharing overlapping workspaces.

In conclusion, solving the CRTSP is not just a thought experiment pursuit but a practical problem for realizing the full potential of robotics for manufacturing. The transition from isolated automation to cooperative systems requires innovative approaches that balance computational efficiency with real-world applicability. By addressing the remaining challenges and refining the methods discussed, CRTSP will enable the next industrial revolution.

Bibliography

- [1] A. Keshvarparast, D. Battini, O. Battaia, and A. Pirayesh, “Collaborative robots in manufacturing and assembly systems: literature review and future research agenda,” *Journal of Intelligent Manufacturing*, vol. 35, no. 5, pp. 2065–2118, Jun. 2024, doi: 10.1007/s10845-023-02137-w[◦].
- [2] P. T. Zacharia, E. K. Xidias, and N. A. Aspragathos, “Task scheduling and motion planning for an industrial manipulator,” *Robotics and Computer-Integrated Manufacturing*, vol. 29, no. 6, pp. 449–462, Dec. 2013, doi: 10.1016/j.rcim.2013.05.002[◦].
- [3] S. Alatartsev, S. Stellmacher, and F. Ortmeier, “Robotic Task Sequencing Problem: A Survey,” *Journal of Intelligent & Robotic Systems*, vol. 80, no. 2, pp. 279–298, Nov. 2015, doi: 10.1007/s10846-015-0190-6[◦].
- [4] C. Wurll, D. Henrich, and H. Wörn, “MULTI-GOAL PATH PLANNING FOR INDUSTRIAL ROBOTS.”
- [5] “Genetic algorithm.” Nov. 03, 2024. Accessed: Dec. 05, 2024. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Genetic_algorithm&oldid=1255083030[◦]
- [6] H. Touzani *et al.*, “Efficient Industrial Solution for Robotic Task Sequencing Problem With Mutual Collision Avoidance & Cycle Time Optimization,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2597–2604, Apr. 2022, doi: 10.1109/LRA.2022.3142919[◦].
- [7] D. Li, H. Su, X. Xu, Q. Wang, J. Qin, and W. Zou, “Cooperative Task Scheduling and Planning Considering Resource Conflicts and Precedence Constraints,” *International Journal of Precision Engineering and Manufacturing*, vol. 24, no. 9, pp. 1503–1516, Sep. 2023, doi: 10.1007/s12541-023-00790-y[◦].
- [8] E. Ablad, D. Spensieri, R. Bohlin, and J. S. Carlson, “Intersection-Free Geometrical Partitioning of Multirobot Stations for Cycle Time Optimization,” *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 842–851, Apr. 2018, doi: 10.1109/TASE.2017.2761180[◦].
- [9] J. Xin, C. Meng, F. Schulte, J. Peng, Y. Liu, and R. R. Negenborn, “A Time-Space Network Model for Collision-Free Routing of Planar Motions in a Multirobot Station,” *IEEE Transactions on Industrial Informatics*, vol. 16, no. 10, pp. 6413–6422, Oct. 2020, doi: 10.1109/TII.2020.2968099[◦].
- [10] D. Spensieri, R. Bohlin, and J. S. Carlson, “Coordination of robot paths for cycle time minimization,” in *2013 IEEE International Conference on Automation Science and Engineering (CASE)*, Madison, WI, USA: IEEE, Aug. 2013, pp. 522–527. doi: 10.1109/CoASE.2013.6654032[◦].

Index of Figures

Figure 1: Overview of Traveling Salesman Problem variations from [3]	7
Figure 2: An example chromosome from [2]	10
Figure 3: Relative CPU time growth from [2]	12
Figure 4: Objective function over a number of generations from [2]. (a) is the B-Spline collision detection while (b) is the bounding box method	12
Figure 5: Working point assignment graph structure	14
Figure 6: Resource allocation graph from [7]	16
Figure 7: Results of the heuristic method in [7].	
(a) CPU runtime versus number of tasks. (b) Cycle time versus number of tasks	18